BOLA: Buffer-Based Near-Optimal Rate Adaptation for Adaptive Bitrate Streaming

Nhung Nguyen
Faculty of Computer Science
University Name
City, Country
email@example.com

***Abstract*—Adaptive bitrate (ABR) streaming algorithms must balance video quality against the risk of playback interruption under variable network conditions. This paper presents a comparative study of two ABR control strategies implemented within a discrete-event simulation (DES) framework: a Predictive Throughput Controller based on harmonic-mean bandwidth estimation, and the Lyapunov Buffer Controller (BOLA), which maximizes a logarithmic utility function subject to playback buffer occupancy constraints. Both controllers are evaluated across nine benchmark scenarios covering stable WiFi, constrained 3G mobile, high-frequency oscillation, and manifest stress conditions. Results show that BOLA consistently outperforms the Predictive Controller in stable and bandwidth-constrained environments, achieving up to 95% higher QoE at tier-boundary operating points, while the Predictive Controller is more robust during abrupt bandwidth collapses. These complementary failure modes motivate a hybrid design that combines throughput estimation during transient phases with Lyapunov optimization at steady state.**

***Index Terms*—adaptive bitrate streaming, MPEG-DASH, BOLA, Lyapunov optimization, quality of experience, buffer-based adaptation**

I. Introduction

II. Background & Related Work

A. MPEG-DASH System Architecture

Dynamic Adaptive Streaming over HTTP (MPEG-DASH), standardized under ISO/IEC 23009-1 [1], establishes a universal framework for streaming multimedia data across unmanaged, heterogeneous IP networks. Unlike traditional stateful streaming protocols that require dedicated network-layer infrastructure, MPEG-DASH relies entirely on the ubiquitous and stateless HTTP/TCP paradigm. By offloading session tracking and delivery logic from media servers to client application layers, the system achieves high scalability, network resilience, and seamless compatibility with standard Content Delivery Networks (CDNs) and proxy caches [3].

The core mechanism of MPEG-DASH is a client-driven pull architecture. Multimedia content is organized using an XML index called the *Media Presentation Description* (MPD), or manifest file [1]. The hierarchical structure is shown in Fig. 1.

As shown in Fig. 1, an MPD contains one or more sequential Periods, each holding AdaptationSets for each media type. Each AdaptationSet offers multiple Representations—encodings of the same content at different bitrates—divided into fixed-duration Segments [2].

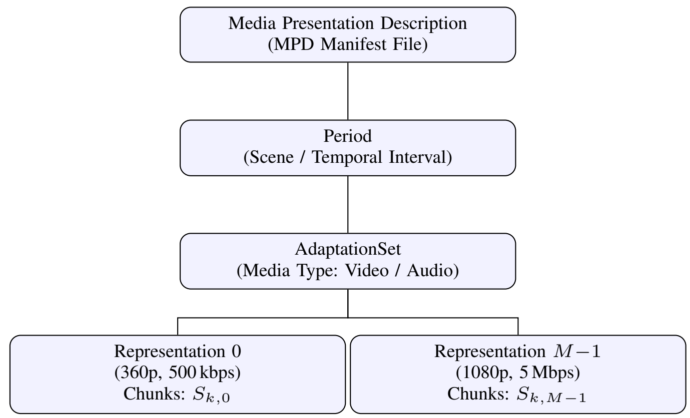

Fig. 1. Hierarchical structure of the MPEG-DASH MPD.

1) Mathematical Abstraction of Server Multimedia Data:

Let $\mathcal{M} = \{0, 1, \ldots, M-1\}$ denote the set of available quality representations. Each tier $m \in \mathcal{M}$ maps to a nominal encoding bitrate:

$$\mathcal{R} = \{R_0, R_1, \ldots, R_{M-1}\}, \quad R_0 < R_1 < \cdots < R_{M-1}. \tag{1}$$

Every segment $k \in \{0, 1, \ldots, N-1\}$ has a fixed playout duration p (seconds). To model Variable Bitrate (VBR) encoding, the file size $S_{k,m}$ (in bytes) is drawn stochastically:

$$\tilde{S}_{k,m} \sim \mathcal{N}(\mu_m, \sigma_m^2), \quad \mu_m = \frac{R_m \cdot p}{8}, \tag{2}$$

with $\sigma_m = 0.20\,\mu_m$ and a hard floor preventing non-positive values:

$$S_{k,m} = \max\Big(\tilde{S}_{k,m},\ 0.10\,\mu_m\Big). \tag{3}$$

B. Heuristic Throughput-Based Approaches

1) Predictive Bandwidth Estimation:

Throughput-based algorithms select the next bitrate tier based on recent download measurements. A well-known example is the FESTIVE framework [3], which assumes short-term capacity correlates with past throughput samples.

Raw wireless measurements fluctuate due to cross-traffic, handovers, and TCP dynamics. A plain arithmetic mean tends to overestimate capacity, causing the client to request a higher tier than the network can sustain.

2) Harmonic Mean Filter: To reduce sensitivity to outliers, a harmonic mean filter is applied over a sliding window of the last $W = 5$ download rates $\mathcal{H}_k = \{\omega_{k-W+1}, \ldots, \omega_k\}$:

$$\hat{\omega}_{k+1} = \frac{W}{\displaystyle\sum_{i=0}^{W-1} \frac{1}{\max(\omega_{k-i},\, 1.0)}}. \tag{4}$$

The $\max(\cdot, 1.0)$ floor prevents division-by-zero during channel blackouts. Because the harmonic mean is sensitive to small values, a single stalled segment heavily penalizes the estimate, yielding a conservative and safe bandwidth prediction.

3) Safety Margin and Rate Selection: A safety coefficient $\rho \in (0, 1]$ (set to $\rho = 0.9$) produces the sustainable throughput threshold:

$$\omega_{\text{safe}} = \hat{\omega}_{k+1} \times \rho. \tag{5}$$

The client selects the highest quality tier satisfying the capacity constraint:

$$m^* = \max\{m \in \mathcal{M} \mid R_m \leq \omega_{\text{safe}}\}. \tag{6}$$

If $\omega_{\text{safe}} < R_0$ or $\mathcal{H}_k = \emptyset$, the algorithm defaults to $m^* = 0$. The complete procedure is summarized in Algorithm 2.

Algorithm 1: Heuristic Throughput-Based Rate Adaptation

Input: History queue $\mathcal{H}_k$, bitrate set $\mathcal{R}$, safety margin ρ
Output: Quality index m^*
if $\mathcal{H}_k = \emptyset$ **then return** 0
$\mathit{SumRec} \leftarrow 0$
for each ω in last W entries of $\mathcal{H}_k$ **do**
    $\mathit{SumRec} \leftarrow \mathit{SumRec} + 1/\max(\omega, 1.0)$
end for
$\hat{\omega}_{k+1} \leftarrow W/\mathit{SumRec}$
$\omega_{\text{safe}} \leftarrow \hat{\omega}_{k+1} \times \rho$
$m^* \leftarrow 0$
for $m = M-1$ **down to** 0 **do**
    if $R_m \leq \omega_{\text{safe}}$ **then**
        $m^* \leftarrow m$; **break**
    end if
end for
return m^*

Fig. 2. Playout adaptation logic based on historic channel state indicators.

C. Buffer-Based Control and Stochastic Optimization

1) Limitations of Throughput-Based Adaptation: Throughput-based controllers try to solve an application-layer problem using noisy network-layer estimates. Buffer-based controllers instead use the client playout buffer occupancy $Q(t)$, which acts as a smoothed proxy for network health. The problem becomes: keep $Q(t)$ within a safe range while maximizing video quality [2].

2) Lyapunov Drift Theory: The buffer-based controller is grounded in the Lyapunov drift-plus-penalty framework for stochastic systems [4]. Define the quadratic Lyapunov function:

$$L(Q(t)) = \frac{1}{2}\left[Q(t) - Q_{\max}\right]^2, \tag{7}$$

and the conditional one-step drift:

$$\Delta L(Q(t)) \triangleq \mathbb{E}\{L(Q(t+1)) - L(Q(t)) \mid Q(t)\}. \tag{8}$$

To maximize user quality satisfaction while bounding underflow risk, the controller minimizes the drift-plus-penalty objective [4]:

$$\min\ \Delta L(Q(t)) - V \cdot \mathbb{E}\{v_m \mid Q(t)\}, \tag{9}$$

where v_m is the perceptual utility of representation m and $V > 0$ trades off quality against buffer safety.

3) The BOLA Optimization Objective: Expanding the drift bound and isolating the controllable decision variable, Spiteri et al. [2] derive a closed-form real-time control law. At each segment download event, the client selects:

$$m^* = \operatorname*{arg\,max}_{m \in \mathcal{M}} \frac{V \cdot (v_m + \gamma) - [Q(t) - \tau]}{S_{k,m}}, \tag{10}$$

where $v_m = \ln(R_m/R_0)$ is the logarithmic perceptual utility, τ is a buffer safety offset, and γ stabilizes quality transitions.

Because (10) depends only on $S_{k,m}$ and $Q(t)$, it runs at $\mathcal{O}(M)$ complexity and bounds time-average performance to within $\mathcal{O}(1/V)$ of the theoretical optimum [2], [4]. Algorithm 3 formalizes the procedure.

Algorithm 2: Lyapunov-Optimized Rate Adaptation (BOLA)

Input: Buffer level $Q(t)$, offset τ, parameters V, γ, segment sizes $\{S_{k,m}\}$, utilities $\{v_m\}$
Output: Quality index m^*
if $Q(t) \leq \tau$ **then return** 0
$\mathit{MaxObj} \leftarrow -\infty$; $m^* \leftarrow 0$
for $m = 0$ **to** $M-1$ **do**
    $\mathit{sz} \leftarrow \max(S_{k,m}, 1.0)$
    $\mathit{Obj} \leftarrow \big[V(v_m + \gamma) - (Q(t) - \tau)\big]/\mathit{sz}$
    if $\mathit{Obj} > \mathit{MaxObj}$ **then**
        $\mathit{MaxObj} \leftarrow \mathit{Obj}$; $m^* \leftarrow m$
    end if
end for
return m^*

Fig. 3. Lyapunov drift minimization loop for real-time buffer-safety optimization.

III. System Design & Architecture

A. Modular Architecture

The framework uses a Discrete-Event Simulation (DES) architecture where the virtual clock jumps between segment download events, ensuring full reproducibility and isolation of adaptation logic from environmental variables.

The design is split into three functional zones:

- Zone 1 (Server Abstraction): a stateless module storing the MPD structure and VBR segment sizes.
- Zone 2 (Network Channel): applies trace-driven bandwidth profiles, jitter, and RTT delays.
- Zone 3 (Adaptive Client Engine): contains the buffer manager, ABR controllers, telemetry logger, and visualization dashboard.

B. The 10-Block Event-Driven Pipeline

The runtime execution loop is decomposed into ten functional blocks, co-operating via localized state-change boundaries as shown in Fig. 4.

The role of each block is as follows:

- Block 1 (MPD Parser): decodes the bitrate set $\mathcal{R}$ and segment parameters.
- Block 2 (VBR Payload Model): generates stochastic segment sizes.
- Block 3 (Channel Interpolator): reads bandwidth traces and outputs $\omega(t)$.
- Block 4 (Virtual Downloader): computes download time $T_k = \text{RTT} + 8S_{k,m}/\omega(t_k)$.
- Block 5 (Buffer Queue): tracks buffer level $Q(t)$.
- Block 6 (Throughput Estimator): applies the harmonic mean filter (4).
- Block 7 (BOLA Optimizer): evaluates the objective (10).
- Block 8 (Rate Arbiter): selects the final quality tier and manages the startup phase.
- Block 9 (Telemetry Logger): records per-segment metrics.
- Block 10 (Dashboard): renders the interactive visualization.

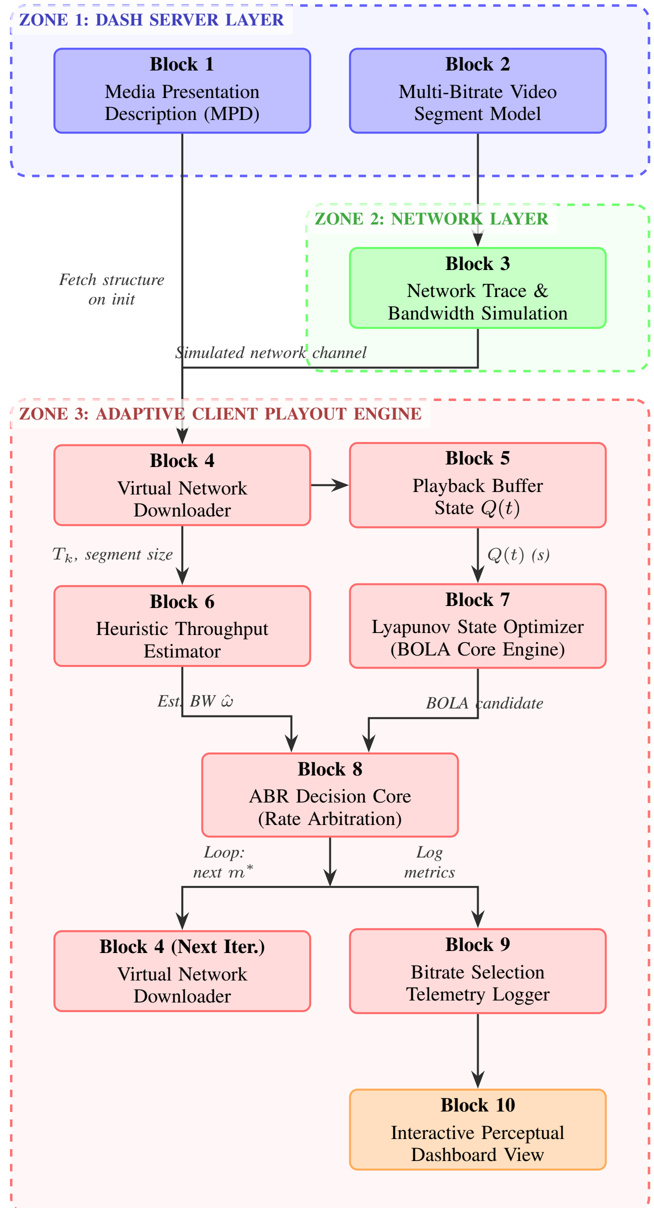

Fig. 4. Structural pipeline across the ten discrete-event simulation blocks.

IV. Methodology & Algorithmic Formulation

A. Discrete-Event Simulation and Playout Dynamics

The runtime loop operates under a Discrete-Event Simulation (DES) model; the virtual clock t advances directly to timestamps of critical events (segment download completions). For segment k, the download duration is:

$$T_k = \text{RTT} + \frac{8\, S_{k,m_k^*}}{\omega(t_k)}, \tag{11}$$

where m_k^* is the selected quality tier and $\omega(t_k)$ is the interpolated channel bandwidth.

Buffer starvation occurs when $T_k > Q_k$, inducing a rebuffering stall:

$$T_{\text{stall},k} = \begin{cases} T_k - Q_k & \text{if } T_k > Q_k, \\ 0 & \text{otherwise.} \end{cases} \tag{12}$$

Upon segment arrival, the buffer updates as:

$$Q_{k+1} = \min(\max(Q_k - T_k,\, 0) + p,\; Q_{\max})\,, \tag{13}$$

and the simulation clock advances by:

$$t_{k+1} = t_k + T_k + T_{\text{stall},k}. \tag{14}$$

B. Hybrid Startup Phase

Pure buffer-based controllers face a cold-start problem: at session start $Q(0) = 0$, equation (10) always returns $m^* = 0$ because $Q(t) \leq \tau$. This gives unnecessarily low quality even on high-capacity links.

To handle this, a startup phase is added: the system uses the throughput-based heuristic until the buffer exceeds $\tau\xi$, then hands control to BOLA. The switching rule is:

$$m_k^* = \begin{cases} \max\{m \mid R_m \leq \hat{\omega}_k \rho\} & \text{if } Q_k \leq \tau\xi \\ & \wedge \text{ StartupMode,} \\ \arg\max_m \dfrac{V(v_m + \gamma) - (Q_k - \tau)}{S_{k,m}} & \text{otherwise.} \end{cases} \tag{15}$$

where $\rho = 0.9$ and $\xi = 1.5$. Once $Q_k > \tau\xi$, *StartupMode* is set to `False` and the Lyapunov controller takes full control.

V. Implementation Details

A. Software Stack

The simulation and visualizer are implemented in Python using:

- NumPy: vectorized computation of VBR payload matrices.
- Pandas: per-segment telemetry logging and CSV export.

B. Visualization Front-End

The user interface uses Streamlit. Session state is stored in `st.session_state` to preserve data across UI interactions. Plotly provides synchronized dual-axis plots of bitrate and buffer level over time.

C. Input Formats

The simulator accepts two input file types:

1) Media manifest (JSON): specifies the bitrate set $\mathcal{R}$, segment duration p, and segment count N.
2) Bandwidth trace (plain text): timestamp–throughput pairs used for piecewise-linear interpolation of $\omega(t)$.

VI. Evaluation Metrics & Setup

A. Quality of Experience (QoE) Metric

Following ITU-T P.1203 [5], the objective QoE score for a session of N segments is:

$$\Psi = \sum_{k=0}^{N-1} \hat{R}_k - \alpha \sum_{k=1}^{N-1} \left| \hat{R}_k - \hat{R}_{k-1} \right| - \beta \sum_{k=0}^{N-1} T_{\text{stall},k}, \tag{16}$$

where $\hat{R}_k = R_{m_k^*}/10^6$ (Mbps), $\alpha = 1.0$ penalizes quality changes, and $\beta = 4.3$ penalizes rebuffering events.

B. Benchmarking Procedure

Both controllers are evaluated under identical conditions using a two-pass procedure:

1) Scan the scenario directory and load each (manifest, trace) pair.
2) Pass 1: run the throughput-based controller and record the segment log.
3) Pass 2: reset all state and run BOLA on the same scenario.
4) Score both logs with (16) and export results as CSV files and comparison plots.

VII. Experimental Results & Analysis

This section reports results from nine network scenarios comparing two controllers: the Predictive Throughput Controller (harmonic-mean estimation) and the Lyapunov Buffer Controller (BOLA). Both are scored using the QoE metric Ψ defined in (16).

Table I maps each descriptive scenario label to its original configuration identifier for cross-reference with the data files.

TABLE I
SCENARIO NAME MAPPING.

Descriptive Name	**Config ID**
Stable WiFi (6 Mbps)	`test_stable_wifi`
Sustained 5 Mbps Flat	`testHD`
Constrained 3G Mobile	`test_low_3g`
V-Shaped Bandwidth Collapse	`testALTsoft`
Square-Wave Oscillation	`test_alt_hard`
Low-BR Manifest + 10 Mbps	`testPQ`
HD Preferred + PQ Trace	`testHDmanPQtrace`
Single-Point Flat (1 Mbps)	`badtest`
Flat 1 Mbps Static Trace	`testALThard`

A. Global Quantitative Summary

Table II consolidates the performance metrics of all nine benchmark scenarios into a single reference. The four evaluation dimensions are: average delivered bitrate, total playout stalls, number of quality-level transitions, and the composite QoE score Ψ.

B. Thematic Clustering Analysis

The nine scenarios are grouped into three clusters based on channel behaviour.

1) Cluster 1 — Baseline High-Bandwidth Channels: Channel: Sustained 5–6 Mbps with low variance ($\sigma < 0.2$ Mbps).

Under stable, high-capacity conditions both controllers reach the highest feasible tier within the first few segments. The hybrid startup phase helps here: while $Q(t) < 3p$, the throughput estimator drives rapid quality ramp-up, avoiding the cold-start delay that pure BOLA would incur.

In the Stable WiFi scenario the Predictive Controller stays at 4.3 Mbps from segment 1, making only 1 quality switch and achieving $\Psi = 120.30$. BOLA reaches a slightly higher average (4.293 Mbps) by climbing to the 6.0 Mbps tier, but the 10 associated switches reduce its QoE to $\Psi = 108.19$.

In the Sustained 5 Mbps Flat scenario the outcome reverses. The Predictive Controller applies its safety margin ($\rho = 0.9$) and settles at 1.0 Mbps, far below the available bandwidth ($\Psi = 27.94$). BOLA detects the growing buffer ($Q(t) > 15\,\text{s}$) and escalates to 5.0 Mbps ($\Psi = 103.93$).

Finding: In stable environments the main differentiator is *convergence strategy*, not stall avoidance. The Predictive Controller favors fewer switches; BOLA favors higher average bitrate.

2) Cluster 2 — Dynamic and Volatile Mobile Channels: Channel: 3G bottleneck (0.5–1.5 Mbps), V-shaped collapse (5 Mbps $\rightarrow$ 100 kbps $\rightarrow$ 5 Mbps), and square-wave oscillation (4 Mbps $\leftrightarrow$ 500 kbps at 1-s intervals).

TABLE II
GLOBAL PERFORMANCE COMPARISON ACROSS ALL NINE BENCHMARK SCENARIOS. BOLD VALUES MARK THE BETTER RESULT PER METRIC. QOE SCORE Ψ FROM (16).

Cluster	Scenario	Metric	Predictive Throughput Controller	Lyapunov Buffer Controller (BOLA)
1. Baseline High-Bandwidth	Stable WiFi (6 Mbps)	Avg Bitrate (Mbps)	4.167	**4.293**
		Playout Stalls (s)	**0.164**	0.165
		Quality Switches	**1**	10
		QoE Score Ψ	**120.30**	108.19
	Sustained 5 Mbps Flat	Avg Bitrate (Mbps)	0.983	**3.917**
		Playout Stalls (s)	**0.247**	0.250
		Quality Switches	**1**	4
		QoE Score Ψ	27.94	**103.93**
2. Dynamic & Volatile Channels	Constrained 3G Mobile	Avg Bitrate (Mbps)	0.550	**0.783**
		Playout Stalls (s)	2.175	**2.095**
		Quality Switches	**2**	7
		QoE Score Ψ	6.15	**10.99**
	V-Shaped Bandwidth Collapse	Avg Bitrate (Mbps)	0.900	**1.433**
		Playout Stalls (s)	**0.256**	4.914
		Quality Switches	**2**	9
		QoE Score Ψ	**24.90**	-3.63
	Square-Wave Oscillation	Avg Bitrate (Mbps)	**1.547**	1.448
		Playout Stalls (s)	5.661	**3.278**
		Quality Switches	**10**	19
		QoE Score Ψ	**13.71**	11.06
3. Manifest & Codec Stress	Low-Bitrate Manifest + 10 Mbps	Avg Bitrate (Mbps)	**0.485**	0.458
		Playout Stalls (s)	**0.059**	0.059
		Quality Switches	**1**	3
		QoE Score Ψ	**13.85**	12.25
	HD Preferred + PQ Trace	Avg Bitrate (Mbps)	**0.485**	0.458
		Playout Stalls (s)	0.060	**0.060**
		Quality Switches	**1**	3
		QoE Score Ψ	**13.84**	12.24
	Single-Point Flat (1 Mbps)	Avg Bitrate (Mbps)	0.500	**0.883**
		Playout Stalls (s)	1.074	**1.024**
		Quality Switches	**0**	3
		QoE Score Ψ	10.38	**20.60**
	Flat 1 Mbps Static Trace	Avg Bitrate (Mbps)	0.500	**0.883**
		Playout Stalls (s)	**1.017**	1.079
		Quality Switches	**0**	3
		QoE Score Ψ	10.63	**20.36**

a) Predictive Controller.: In the Constrained 3G Mobile scenario the controller stays at 0.5 Mbps throughout to avoid buffer underflow ($\Psi = 6.15$). In the V-Shaped Collapse scenario it recovers quickly once bandwidth returns, achieving $\Psi = 24.90$ with only 0.256 s of stall.

b) BOLA.: In the 3G scenario BOLA raises quality to 1.0 Mbps when buffer headroom allows, reaching an average of 0.783 Mbps and $\Psi = 10.99$—a 79% gain over the Predictive Controller.

In the V-Shaped Collapse BOLA's buffer-centric logic becomes a problem. The 100 kbps trough empties the buffer; after recovery BOLA must rebuild $Q(t)$ above τ before it allows higher bitrates, producing 4.914 s of stall ($\approx 20\times$ more than the Predictive Controller) and $\Psi = -3.63$.

In the Square-Wave scenario the Predictive Controller reacts faster ($\Psi = 13.71$ vs. 11.06), but BOLA has fewer total stalls (3.278 s vs. 5.661 s) with more quality switches (19 vs. 10).

Finding: The two controllers have opposite failure modes. The Predictive Controller is too conservative under sustained low bandwidth; BOLA suffers from hysteresis during bandwidth recovery. This supports hybrid designs that switch between throughput estimation and Lyapunov control depending on network state.

3) Cluster 3 — Manifest Stress and Edge Cases: Channel: Low-bitrate manifests (50–500 kbps) against a 10 Mbps link; and single-point flat traces.

In the Low-Bitrate Manifest + 10 Mbps and HD Preferred + PQ Trace scenarios the channel (10 Mbps) is $20\times$ the manifest ceiling (500 kbps). Both controllers saturate the highest tier almost immediately and produce near-identical results ($\Psi \approx 13.8$ vs. ≈ 12.2). The small gap comes only from BOLA's 3 exploratory switches at startup vs. the Predictive Controller's 1. Setting `Preferred_Bitrate` to an out-of-range value (5 Mbps) in the HD Preferred scenario has no effect on either controller.

In the Single-Point Flat and Static Trace scenarios (both at 1 Mbps) the Predictive Controller computes a safe threshold of $0.9 \times 1.0 = 0.9$ Mbps, which is below the 1.0 Mbps tier. It therefore stays at 0.5 Mbps for the whole session ($\Psi \approx 10.5$). BOLA notices that the buffer keeps growing at 1.0 Mbps and selects that tier whenever $Q(t)$ permits, reaching an average of 0.883 Mbps and $\Psi \approx 20.5$ (a 95% gain). Results are virtually identical between the two scenarios, confirming that trace density does not change this boundary-bias finding.

Finding: When available bandwidth sits exactly at a tier boundary, the Predictive Controller's safety margin forces a systematic downward bias. BOLA avoids this by using segment-level buffer feasibility rather than a global throughput estimate.

C. Selected Visual Evidence

Three representative scenarios are shown below. Each plot has two columns (Predictive left, BOLA right) and two rows (bitrate top, buffer bottom). The remaining six plots appear in Appendix A.

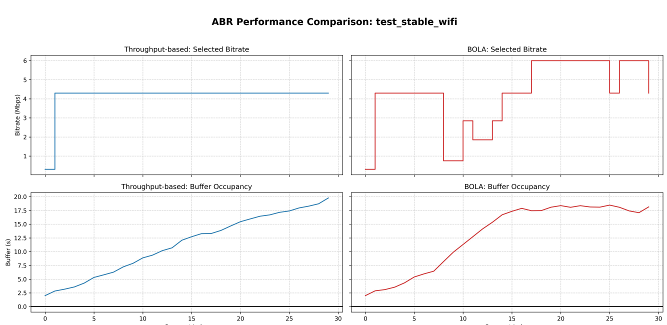

Fig. 5. Stable WiFi (6 Mbps). Top: bitrate vs. segment index; Predictive settles at 4.3 Mbps (1 switch), BOLA reaches 6.0 Mbps (10 switches). Bottom: both controllers hold $Q(t) > 15$ s with no stalls.

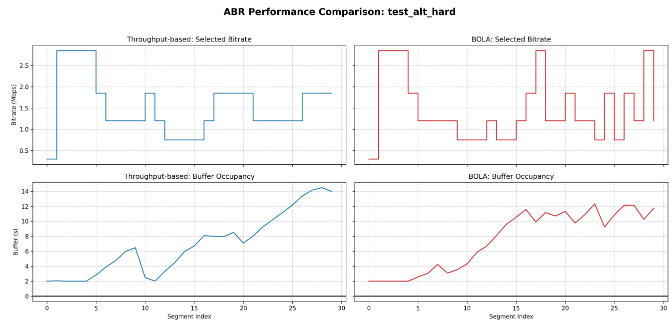

Fig. 6. Square-Wave Oscillation (4 Mbps $\leftrightarrow$ 500 kbps). Top: Predictive shows sharp tier transitions; BOLA shows frequent small oscillations. Bottom: BOLA keeps a more stable buffer while Predictive dips lower during low-bandwidth phases.

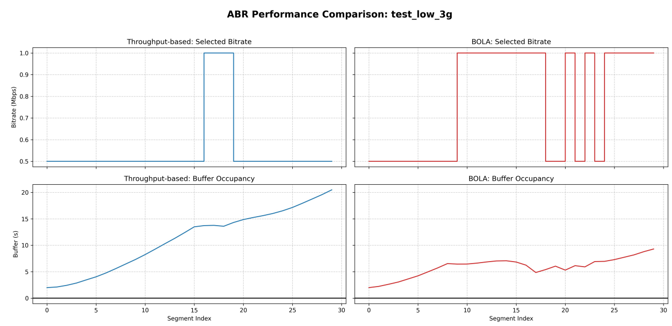

Fig. 7. Constrained 3G Mobile (0.5–1.5 Mbps). Top: Predictive stays at 0.5 Mbps; BOLA escalates to 1.0 Mbps when buffer permits. Bottom: both controllers struggle to keep $Q(t) > 3$ s.

VIII. Conclusion

This paper compared two ABR streaming controllers—a harmonic-mean Predictive Controller and the Lyapunov-optimized BOLA—across nine network scenarios using a reproducible discrete-event simulation framework.

Three consistent behavioral patterns emerge from the experiments. In stable, high-bandwidth environments both controllers perform adequately, but with different trade-offs: the Predictive Controller achieves higher QoE through quality stability (fewer switches), while BOLA achieves higher average bitrate through opportunistic tier escalation. Under volatile mobile conditions the Predictive Controller recovers faster from bandwidth collapses due to its direct throughput tracking, yet remains overly conservative under sustained low bandwidth. BOLA's buffer-centric approach excels at exploiting moderate bandwidth headroom but suffers from hysteresis after severe buffer drain events. At tier boundaries, BOLA's segment-level feasibility check avoids the systematic downward bias caused by the Predictive Controller's fixed safety margin, yielding up to a 95% QoE improvement in those cases.

These results confirm that neither strategy dominates across all conditions, and that the two controllers have complementary strengths. The Predictive Controller is preferable when the network is highly volatile or subject to sharp collapses;

BOLA is preferable under stable or moderately constrained conditions. Future work will investigate adaptive hybrid controllers that dynamically select the estimation strategy based on detected network state, and extend the evaluation to real-world streaming traces under concurrent traffic loads.

References

[1] ISO/IEC 23009-1:2022, *Information technology—Dynamic adaptive streaming over HTTP (DASH)—Part 1: Media presentation description and segment formats*, 2022.

[2] K. Spiteri, R. Urgaonkar, and R. K. Sitaraman, "BOLA: Near-optimal bitrate adaptation for online videos," *IEEE/ACM Trans. Netw.*, vol. 28, no. 4, pp. 1698–1711, Aug. 2020.

[3] J. Jiang, V. Sekar, and H. Zhang, "Improving fairness, efficiency, and stability in HTTP-based adaptive video streaming with FESTIVE," in *Proc. ACM CoNEXT*, Nice, France, Dec. 2012, pp. 97–108.

[4] M. J. Neely, *Stochastic Network Optimization with Application to Communication and Queueing Systems*. San Rafael, CA: Morgan & Claypool, 2010.

[5] ITU-T Recommendation P.1203, *Parametric bitstream-based quality assessment of progressive download and adaptive audiovisual streaming services over reliable transport*, Oct. 2017.

Appendix A
Supplementary Comparison Plots

The remaining six scenarios are shown below for completeness.

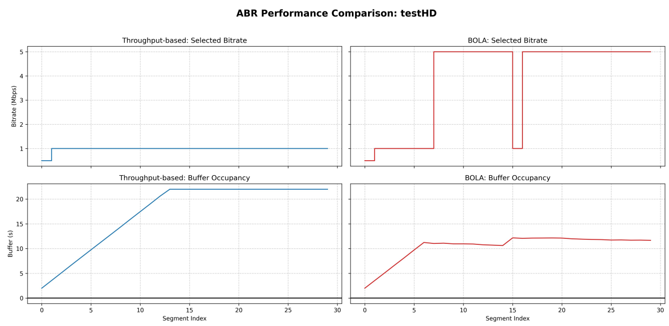

Fig. 8. Sustained 5 Mbps Flat — BOLA reaches 5.0 Mbps; Predictive under-selects at 1.0 Mbps.

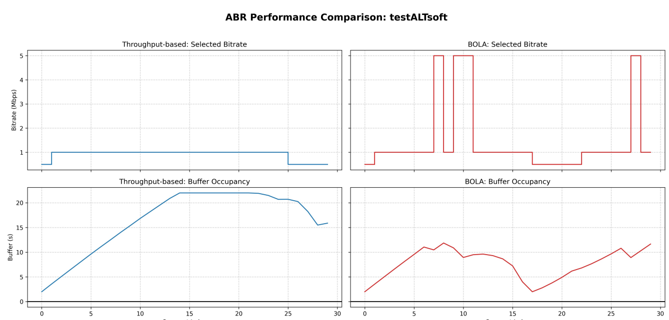

Fig. 9. V-Shaped Collapse — BOLA: 4.914 s stall; Predictive: 0.256 s stall.

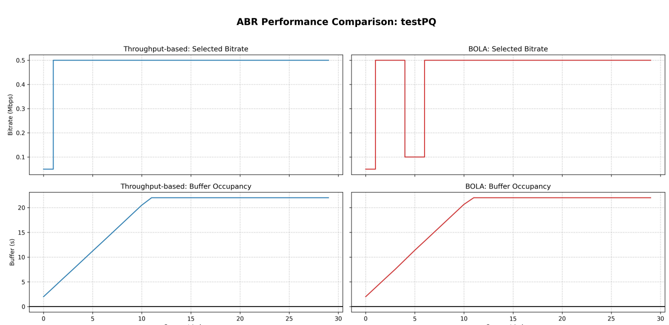

Fig. 10. Low-BR Manifest + 10 Mbps — both converge immediately; BOLA makes 3 startup switches.

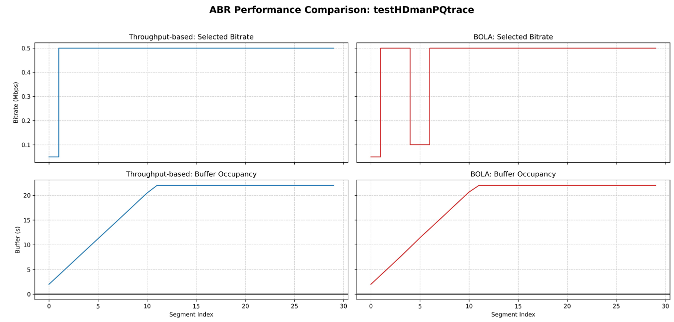

Fig. 11. HD Preferred + PQ Trace — out-of-range `Preferred_Bitrate` has no effect.

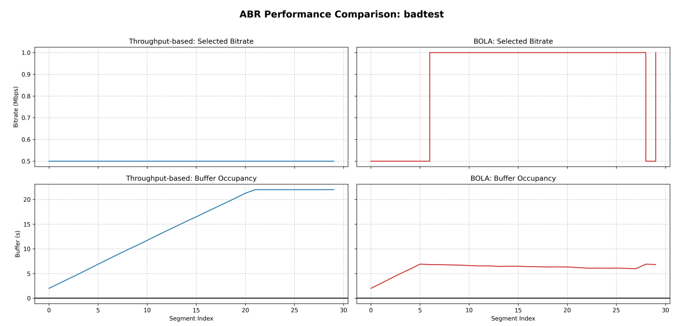

Fig. 12. Single-Point Flat (1 Mbps) — Predictive: 0.5 Mbps; BOLA: 0.883 Mbps ($\Psi = 20.60$ vs. 10.38).

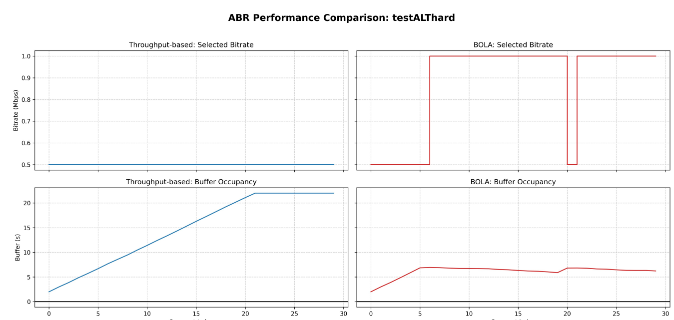

Fig. 13. Flat 1 Mbps Static Trace — five entries, all at 1 Mbps; outcome matches Single-Point Flat.